23CSE455 Design Patterns
Case Study Report

Item	Content
Team	A15
Case study	Design-pattern discovery, evolution and cross-language comparison of a Quarkus (Java) bank REST API; Title: Design Pattern Discovery and Evolution in a Quarkus Banking REST API (Java; cross-language comparison with Python, JavaScript and C++).
Application	api-quarkus-2 (“REST API Bank - Quarkus”); faculty approval: _____(name / date, to be signed)
Date	2026-10-11
Submission repository	23CSE455_DP_CaseStudy, team folder at the repository root

Roll number	Student name	Batch
AM.AI.U4AID23009	Hariprid V	AIDS
AM.AI.U4AID23017	Pranav R Menon	AIDS
AM.AI.U4AID23022	Sreyas A	AIDS

Contents

1 Application and architecture 3

1.1 Architecture 3

2 Part 1: Pattern discovery 4

2.1 Totals at the baseline 4

2.2 Pattern inventory 4

2.3 One record for each pattern instance 5

2.3.1 Account and User Persistence DAO 5

2.3.2 API Payload DTO Assembly 6

2.3.3 Banking Use-Case Service Layer 7

2.3.4 Account Deposit and Withdrawal Script 8

2.3.5 REST Use-Case Controller 9

2.3.6 CDI Field Injection Wiring 10

2.3.7 HTTP Error Translation 10

3 Part 2: Pattern evolution 11

3.1 Counts per source version 11

3.2 CH01 - Account-type policies (overdraft for checking, exact-balance withdrawal) 12

3.3 CH02 - Transaction events: audit log and high-value alert observers 13

3.4 CH03 - Atomic funds transfer; business rules move into Account 14

3.5 Status of every instance across versions 15

4 Part 3: Cross-language comparison 16

4.1 Selected instances and common specification 16

4.2 Results 16

5 LLM usage records **18**

6 Findings, contributions and references **19**

6.1 Findings 19

6.2 Contributions 20

6.3 References 20

7 Team folder and repository **20**

1 Application and architecture

Item	Content
Original source	Repository URL: https://github.com/dev-MichaelCruz/api-quarkus; supplied as the archive api-quarkus-main.zip (no tag or version string inside). Baseline commit: baa96db = local import commit of the archive; the only edit is that the plain-text database password in application.properties was replaced by ${DB_PASSWORD:change-me} (never upload credentials). Access date 2026-10-11. License: none stated (no LICENSE file in the archive).
Environment	Project: Java release 17, Quarkus platform 3.6.1 (extensions: arc, hibernate-orm, hibernate-validator, resteasy-jackson, jdbc-mysql; junit5 and rest-assured for tests), MySQL. Working container: OpenJDK 21.0.12.1, Python 3.13.16, Node.js 22.22.0, g++ 13.3.0, lizard 1.24.1, Graphviz 2.43.0, pdfTeX 2023.
Scope	Inspected: all 17 production Java files (613 lines), application.properties, pom.xml. Excluded: Dockerfiles and index.html (no application logic), Maven wrapper scripts (tooling), and the template tests GreetingResourceTest/IT (they request / hello, which does not exist in src/main, so by inspection they cannot pass; removed in commit c9d7097).
Run instructions	README: data/code/README.md (project) and data/code/api-quarkus-2/README.md: ./mvnw compile quarkus:dev with a MySQL database configured through DB_PASSWORD. **Actual result: not run.** Maven Central is unreachable from the working environment (proxy answers HTTP 403 to CONNECT), so no Quarkus build, mvn test or database run was possible. Substitute actually run: sandbox harness (real production sources compiled against Jakarta API stubs, in-memory EntityManager): bash data/test result/harness/run_tests.sh – 15/15 characterization tests passed on the unchanged baseline production code.

1.1 Architecture

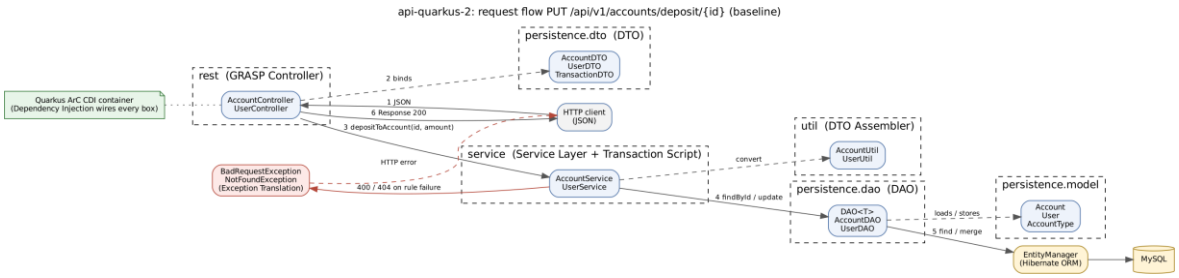


Figure 1: Layered architecture and the request flow of PUT /api/v1/accounts/deposit/{id} (baseline). Source: Part1/pattern_architecture/architecture.dot.

The application is a classic layered REST service. **Components:** controllers (rest) bind JSON to DTOs and delegate; services (service) hold the use cases and the business rules; converters (util) map DTO and entity; DAOs (persistence.dao) wrap the JPA EntityManager; entities and DTOs live in persistence.model and persistence.dto; two WebApplicationException subclasses carry HTTP errors; the CDI container wires everything with @Inject. **Request flow (deposit):** (1) the client sends JSON; (2) RESTEasy binds it to TransactionDTO and calls AccountController.depositToAccount; (3) the controller unwraps the amount and calls AccountService.depositToAccount; (4) the service loads the account through AccountDAO.findById (404 if absent), checks the amount, applies the savings bonus, and saves through AccountDAO.update; (5) the DAO calls entityManager.merge; (6) the controller answers 200. A rule violation throws BadRequestException (HTTP 400) which the JAX-RS runtime turns into the response.

2 Part 1: Pattern discovery

Method. Each member ran a prompt restricted to one layer (presentation/contract, business, persistence; files Part1/Part1_prompt.txt, Part1/llm/) over the numbered baseline source. The team merged the three outputs, removed duplicates (the DTO and its assembler were seen from two sides) and decided rejections. **Source evidence checked:** (a) every ROLE / symbol / path:start-end row of the three LLM outputs (68 rows) was compared by script with the real declaration range – 68/68 matched (Part1/evidence/llm_line_check.txt); (b) every behavioural claim was checked with greps (Part1/evidence/claims_check.txt) and by the characterization tests; one LLM claim was wrong and is corrected (Section 5); (c) the line ranges in patterns.csv are computed from the source by javaloc.py, not typed. Observation scope: all production sources and application. properties at commit baa96db.

2.1 Totals at the baseline

Measure	Value
Confirmed pattern types	7: Data Access Object, Data Transfer Object, Service Layer, Transaction Script, Controller, Dependency Injection, Exception Translation
Confirmed pattern instances	7 (one independent collaboration per type)
Concrete participant classes	21 (distinct classes realising the variant role of each instance: DAO 2, DTO 3, Service Layer 2, Transaction Script 2, Controller 2, Dependency Injection 8, Exception Translation 2; a class serving two instances is counted in each)
Unique participant symbols	36 distinct classes/interfaces/methods (shared symbols counted once); 11 injection-point fields are mapped additionally
Rejected and uncertain candidates	10 rejected, 3 uncertain (reasons below); scope: whole baseline production source

2.2 Pattern inventory

Pattern type	Instance name	Instances	Concrete classes
Data Access Object	Account and User Persistence DAO	1	2
Data Transfer Object	API Payload DTO Assembly	1	3
Service Layer	Banking Use-Case Service Layer	1	2
Transaction Script	Account Deposit and Withdrawal Script	1	2
Controller	REST Use-Case Controller	1	2
Dependency Injection	CDI Field Injection Wiring	1	8
Exception Translation	HTTP Error Translation	1	2

Rejected candidates

Candidate	Raised by	Reason
Builder (Response.status().entity().build())	Member 1	Builder is implemented by JAX-RS; controllers are only clients.
Front Controller	Member 1	No dispatcher class in the supplied code; dispatching is framework-internal.
Strategy (savings bonus)	Member 2	Single if on an enum value at AccountService 46-48; no strategy interface or interchangeable family.
Factory Method / Simple Factory (AccountUtil.convertToAccount)	Member 2	Field mapping only; no product hierarchy or creation choice.
Adapter (*Util classes)	Member 2	They convert data, they do not adapt an incompatible interface.

Candidate	Raised by	Reason
GoF Singleton (@Application-Scoped beans)	Member 2	No private constructor / static instance; lifetime is container-managed (covered by Dependency Injection).
Active Record	Member 3	Account and User contain no persistence behaviour.
Repository as separate instance	Member 3	Same collaboration as the DAO under another name; classes expose save/update, not collection-like add/remove.
Null Object (Optional from findById)	Member 3	Optional.of(null) throws NullPointerException; no neutral object exists.
Layered Architecture as separate instance	Team (integration)	Would double-count Controller + Service Layer + DAO, which are already counted; used as the architecture description instead.

Uncertain candidates

Candidate	Raised by	Reason it stays uncertain
Facade (AccountService.createAccount)	Member 2	Hides three collaborators behind one call, but that is the ordinary job of a service method already counted as Service Layer.
Data Mapper (JPA EntityManager / Hibernate)	Member 3	The mapper is inside the persistence provider; application code only supplies mapping annotations.
Interceptor via @Transactional (AccountDAO 30/35, UserDAO 28)	Member 3	Annotation-only evidence; interceptor lives in a dependency and was not executed in the sandbox.

2.3 One record for each pattern instance

2.3.1 Account and User Persistence DAO

Pattern type and category: Data Access Object – Enterprise (Core J2EE). **Purpose and origin:** Keep persistence calls (find/persist/merge/query) out of the business layer behind a small entity-oriented API. Origin: application code.

Pattern role	Actual class / interface / function	File and line range
DataAccessObject	DAO	persistence/dao/DAO.java:6-11
ConcreteDAO	AccountDAO	persistence/dao/AccountDAO.java:13-45
ConcreteDAO	UserDAO	persistence/dao/UserDAO.java:13-38
DataSource	AccountDAO.entityManager	persistence/dao/AccountDAO.java:16-17
DataSource	UserDAO.entityManager	persistence/dao/UserDAO.java:16-17
TransferObject	Account	persistence/model/Account.java:5-44
TransferObject	User	persistence/model/User.java:5-56
BusinessObject	AccountService	service/AccountService.java:18-85
BusinessObject	UserService	service/UserService.java:15-46
Operation	AccountDAO.findById	persistence/dao/AccountDAO.java:19-28
Operation	AccountDAO.save	persistence/dao/AccountDAO.java:30-33
Operation	AccountDAO.update	persistence/dao/AccountDAO.java:35-39
Operation	AccountDAO.getAll	persistence/dao/AccountDAO.java:41-44
Operation	UserDAO.findById	persistence/dao/UserDAO.java:19-26
Operation	UserDAO.save	persistence/dao/UserDAO.java:28-32
Operation	UserDAO.getAll	persistence/dao/UserDAO.java:34-37

Evidence: AccountDAO/UserDAO wrap entityManager.find/persist/merge/createQuery; services call only DAO methods and never reference EntityManager (grep C2); DAO<T> declares the shared findById/getAll contract, but save/update exist only on the concrete DAOs, so services inject the concrete classes (weakened abstraction). Tests replace the EntityManager by an in-memory fake without touching production code.

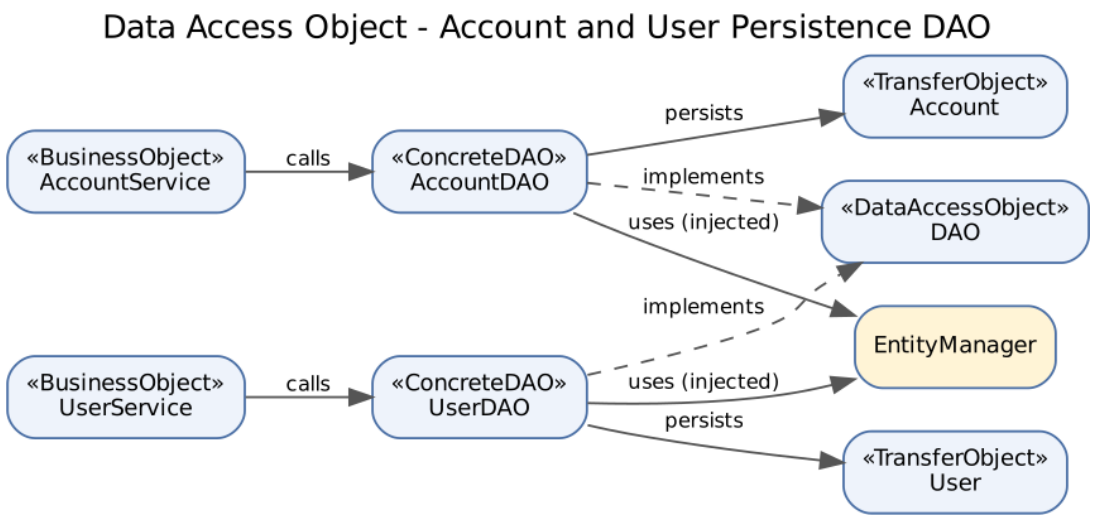


Figure 2: Account and User Persistence DAO – class-level diagram (editable source Part1/pattern_architecture/instances/DAO.dot).

Verification: Confirmed. Reviewer: Hariprid V, Pranav R Menon, Sreyas A. Justification and tests: BaselineBehaviourTest: createUserStoresAllFields, createAccountLinksUserAndKeepsBalance, listAccountsReturnsAllAccounts (all persistence goes through the DAOs onto an in-memory EntityManager); greps C2, C6, C10. Alternatives considered: Repository (rejected as a separate instance: same collaboration, DAO naming and save/update API).

2.3.2 API Payload DTO Assembly

Pattern type and category: Data Transfer Object – Enterprise (Fowler PoEAA / Core J2EE). **Purpose and origin:** Decouple the JSON request contract from the persistence entities and carry only the needed fields across the REST boundary. Origin: application code.

Pattern role	Actual class / interface / function	File and line range
DataTransferObject	UserDTO	persistence/dto/UserDTO.java:5-60
DataTransferObject	AccountDTO	persistence/dto/AccountDTO.java:6-50
DataTransferObject	TransactionDTO	persistence/dto/TransactionDTO.java:5-15
Assembler	UserUtil.convertToUserDTO	util/UserUtil.java:18-28
Assembler	AccountUtil.convertToAccount	util/AccountUtil.java:21-33
Assembler	UserService.createUser	service/UserService.java:25-34
DomainObject	User	persistence/model/User.java:5-56
DomainObject	Account	persistence/model/Account.java:5-44
Consumer	AccountController.createAccount	rest/AccountController.java:21-26
Consumer	AccountController.depositToAccount	rest/AccountController.java:28-34
Consumer	AccountController.withdraw	rest/AccountController.java:36-42
Consumer	UserController.createUser	rest/UserController.java:16-20

Evidence: The three DTO classes hold fields, validation annotations and accessors only; controller methods take DTOs as parameters; convertToUserDTO, convertToAccount and UserService.createUser copy field-by-field between DTO and entity. Partial: listAccounts returns Account entities directly (output side has no DTO).

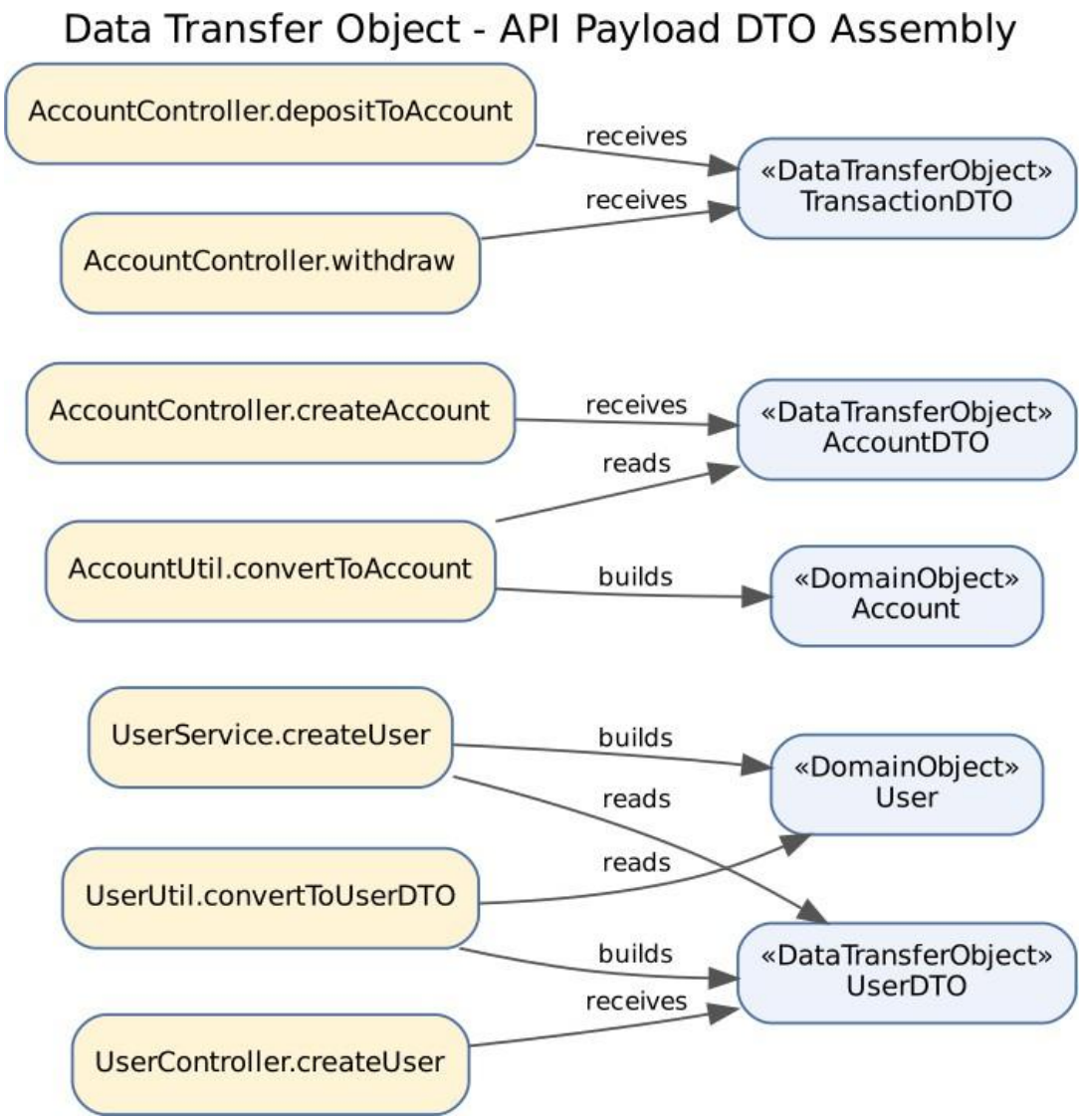


Figure 3: API Payload DTO Assembly – class-level diagram (editable source Part1/pattern_architecture/instances/DTO.dot).

Verification: Confirmed. Reviewer: Hariprid V, Pranav R Menon, Sreyas A. Justification and tests: BaselineBehaviourTest.createUserStoresAllFields (DTO -> entity), deposit/withdraw tests (TransactionDTO); grep C5. Partial: listAccounts returns entities (output side not protected). Alternatives considered: Adapter / Factory for the util classes (rejected: they copy data, they do not adapt an interface or choose a product).

2.3.3 Banking Use-Case Service Layer

Pattern type and category: Service Layer – Enterprise (Fowler PoEAA). **Purpose and origin:** Provide one application boundary that exposes the use cases and coordinates DAOs and converters for the controllers. Origin: application code.

Pattern role	Actual class / interface / function	File and line range
ServiceLayer	AccountService	service/AccountService.java:18-85
ServiceLayer	UserService	service/UserService.java:15-46
Client	AccountController	rest/AccountController.java:13-50
Client	UserController	rest/UserController.java:10-21
Operation	AccountService.createAccount	service/AccountService.java:30-34
Operation	AccountService.depositToAccount	service/AccountService.java:36-58

Pattern role	Actual class / interface / function	File and line range
Operation	AccountService.withdraw	service/AccountService.java:60-79
Operation	AccountService.listAllAccounts	service/AccountService.java:81-83
Operation	UserService.createUser	service/UserService.java:25-34
Operation	UserService.getUserDetails	service/UserService.java:36-44
Collaborator	AccountService.accountDAO	service/AccountService.java:20-21
Collaborator	AccountService.userService	service/AccountService.java:23-24
Collaborator	AccountService.accountUtil	service/AccountService.java:26-27
Collaborator	UserService.userDAO	service/UserService.java:18-19
Collaborator	UserService.userUtil	service/UserService.java:21-22

Evidence: Controllers call only AccountService/UserService; AccountService.createAccount coordinates userService.getUserDetails, accountUtil.convertToAccount and accountDAO.save. Deviation: transaction demarcation is on DAO methods, not on the service operations.

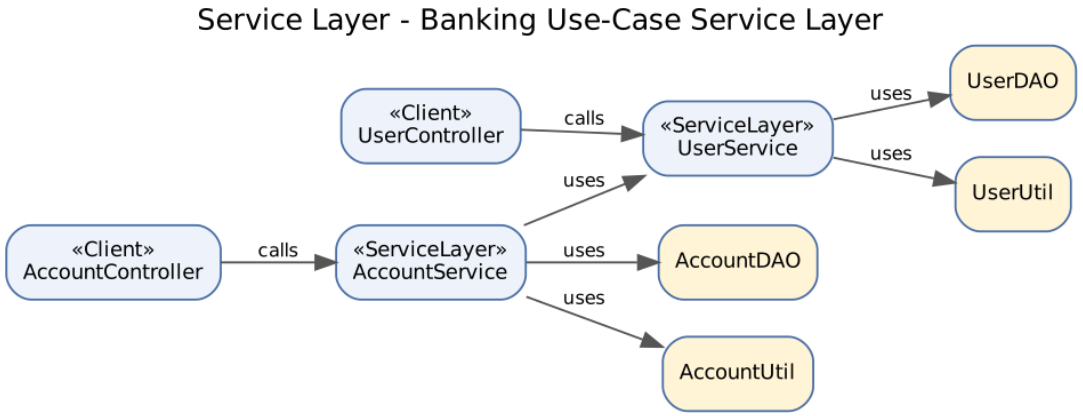


Figure 4: Banking Use-Case Service Layer – class-level diagram (editable source Part1/pattern_architecture/instances/SL.dot).

Verification: Confirmed. Reviewer: Hariprid V, Pranav R Menon, Sreyas A. Justification and tests: All BaselineBehaviourTest flows run controller -> service -> DAO; grep C5. Alternatives considered: Facade for AccountService.createAccount (uncertain: ordinary service job already counted here).

2.3.4 Account Deposit and Withdrawal Script

Pattern type and category: Transaction Script – Enterprise (Fowler PoEAA). **Purpose and origin:** Implement each use case as one procedure that applies the business rules directly to row-like data. Origin: application code.

Pattern role	Actual class / interface / function	File and line range
TransactionScript	AccountService.depositToAccount	service/AccountService.java:36-58
TransactionScript	AccountService.withdraw	service/AccountService.java:60-79
TransactionScript	AccountService.createAccount	service/AccountService.java:30-34
TransactionScript	UserService.createUser	service/UserService.java:25-34
RowData	Account	persistence/model/Account.java:5-44

Evidence: Rules are inline in the service methods: amount > 0 (AccountService line 43), savings bonus += 0.5 (lines 46-48), withdraw allowed only if amount > 0 && balance > amount (line 67). Account contains only accessors (grep C9), i.e. an anemic entity.



Figure 5: Account Deposit and Withdrawal Script – class-level diagram (editable source Part1/pattern_architecture/instances/TS.dot).

Verification: Confirmed. Reviewer: Hariprid V, Pranav R Menon, Sreyas A. Justification and tests: depositToSavingsAddsAmountPlusHalfBonus and withdrawOfExactlyTheBalanceIsRejected-InBaseline pin the rules that live in the service; greps C8, C9 (Account has accessors only). Alternatives considered: Domain Model (absent at baseline because Account has no behaviour; introduced in CH03).

2.3.5 REST Use-Case Controller

Pattern type and category: Controller – GRASP. **Purpose and origin:** Receive HTTP system events and hand them to the business layer without business logic in the HTTP layer. Origin: application code.

Pattern role	Actual class / interface / function	File and line range
Controller	AccountController	rest/AccountController.java:13-50
Controller	UserController	rest/UserController.java:10-21
SystemOperation	AccountController.createAccount	rest/AccountController.java:21-26
SystemOperation	AccountController.depositToAccount	rest/AccountController.java:28-34
SystemOperation	AccountController.withdraw	rest/AccountController.java:36-42
SystemOperation	AccountController.listAccounts	rest/AccountController.java:44-48
SystemOperation	UserController.createUser	rest/UserController.java:16-20
Delegate	AccountController.accountService	rest/AccountController.java:16-17
Delegate	UserController.userService	rest/UserController.java:13-14

Evidence: Each controller method unwraps the DTO, makes exactly one service call and wraps the result in a Response (grep C5); no DAO or EntityManager reference in rest/ (grep C1).

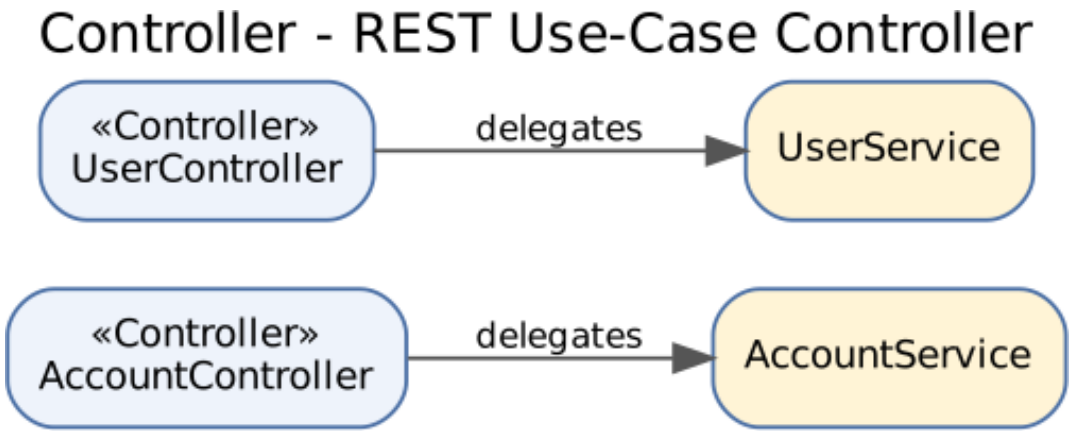


Figure 6: REST Use-Case Controller – class-level diagram (editable source Part1/pattern_architecture/instances/CTRL.dot).

Verification: Confirmed. Reviewer: Hariprid V, Pranav R Menon, Sreyas A. Justification and tests: createAccountEndpointReturns201, listAccountsReturnsAllAccounts; greps C1, C5, C12. Alternatives considered: Front Controller (rejected: no dispatcher code in the application).

2.3.6 CDI Field Injection Wiring

Pattern type and category: Dependency Injection – Framework idiom (CDI / IoC). **Purpose and origin:** Let the container create and wire collaborators so that classes depend on types, not on construction. Origin: framework idiom (Jakarta CDI / Quarkus ArC) used by the application.

Pattern role	Actual class / interface / function	File and line range
Container	(Quarkus ArC CDI container - dependency, not in application source)	not in application source
InjectionPoint	AccountController.accountService	rest/AccountController.java:16-17
InjectionPoint	UserController.userService	rest/UserController.java:13-14
InjectionPoint	AccountService.accountDAO	service/AccountService.java:20-21
InjectionPoint	AccountService.userService	service/AccountService.java:23-24
InjectionPoint	AccountService.accountUtil	service/AccountService.java:26-27
InjectionPoint	UserService.userDAO	service/UserService.java:18-19
InjectionPoint	UserService.userUtil	service/UserService.java:21-22
InjectionPoint	AccountDAO.entityManager	persistence/dao/AccountDAO.java:16-17
InjectionPoint	UserDAO.entityManager	persistence/dao/UserDAO.java:16-17
InjectionPoint	AccountUtil.accountDAO	util/AccountUtil.java:17-18
InjectionPoint	UserUtil.userDAO	util/UserUtil.java:14-15
ManagedBean	AccountController	rest/AccountController.java:13-50
ManagedBean	UserController	rest/UserController.java:10-21
ManagedBean	AccountService	service/AccountService.java:18-85
ManagedBean	UserService	service/UserService.java:15-46
ManagedBean	AccountUtil	util/AccountUtil.java:14-35
ManagedBean	UserUtil	util/UserUtil.java:12-30
ManagedBean	AccountDAO	persistence/dao/AccountDAO.java:13-45
ManagedBean	UserDAO	persistence/dao/UserDAO.java:13-38

Evidence: No collaborator is created with new (grep C3 lists only data objects and exceptions); every collaborator is an @Inject field; the unit tests build the same classes in a MiniContainer and inject an in-memory EntityManager without changing production code. Dead injection points: AccountUtil.accountDAO and UserUtil.userDAO are never used (grep C4).

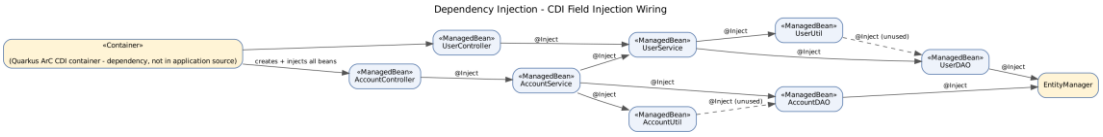


Figure 7: CDI Field Injection Wiring – class-level diagram (editable source Part1/pattern_architecture/instances/DI.dot).

Verification: Confirmed. Reviewer: Hariprid V, Pranav R Menon, Sreyas A. Justification and tests: Every test builds the real classes in a MiniContainer that injects the in-memory EntityManager into the DAOs without changing production code; greps C3, C4. Alternatives considered: GoF Singleton for the @ApplicationScoped beans (rejected: no private constructor or static instance).

2.3.7 HTTP Error Translation

Pattern type and category: Exception Translation – Framework idiom (JAX-RS). **Purpose and origin:** Convert business failures into HTTP 400/404 responses with a message body. Origin: framework idiom (JAX-RS) used by the application.

Pattern role	Actual class / interface / function	File and line range
TranslatedException	BadRequestException	exception/BadRequestException.java:6-11
TranslatedException	NotFoundException	exception/NotFoundException.java:6-11
Raiser	AccountService.depositToAccount	service/AccountService.java:36-58

Pattern role	Actual class / interface / function	File and line range
Raiser	AccountService.withdraw	service/AccountService.java:60-79
Raiser	AccountDAO.findById	persistence/dao/AccountDAO.java:19-28
Raiser	UserService.getUserDetails	service/UserService.java:36-44
Translator	(JAX-RS runtime - dependency, not in application source)	not in application source

Evidence: Both exceptions extend WebApplicationException and build Response(status, message) in the constructor; service/DAO code throws them; unit tests assert getResponse().getStatus() 400/404 and the entity text. The runtime half (JAX-RS turning the exception into the HTTP reply) is framework behaviour and was not executed in the sandbox.

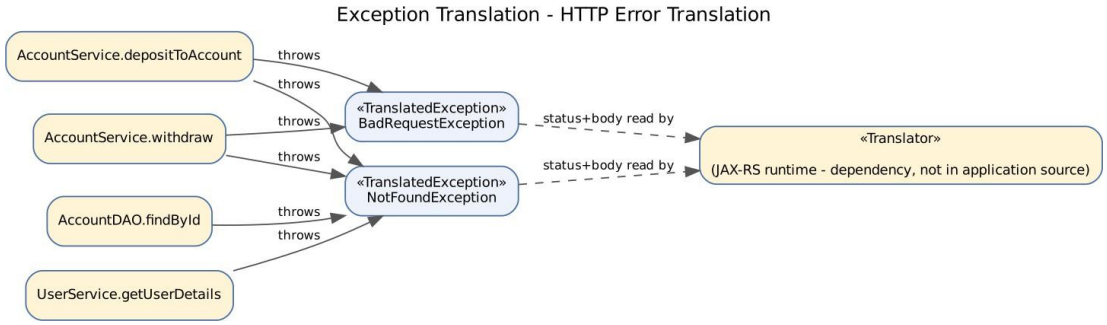


Figure 8: HTTP Error Translation – class-level diagram (editable source Part1/pattern_\ architecture/ instances/ET.dot).

Verification: Confirmed. Reviewer: Hariprid V, Pranav R Menon, Sreyas A. Justification and tests: errorsAreCarriedAsHttpResponsesByTheCustomExceptions and the 400/404 assertions in the deposit/withdraw tests. The runtime half (JAX-RS mapping the exception to the HTTP reply) is specified by Jakarta REST but was not executed here. Alternatives considered: A separate ExceptionMapper (none exists in the application; the response is embedded in the exception).

3 Part 2: Pattern evolution

Three business changes were implemented one after another on the real code (git history in the local working repository; every change has a patch, a requirement file, acceptance criteria and a test log). Versions: Original = baa96db (+ c9d7097 test scaffolding, production code unchanged); Change 1 = 6bdf74e; Change 2 = c755d20; Change 3 = b8de930. Each version is counted separately, and instances *replaced* by a change are excluded from the totals of the later version.

3.1 Counts per source version

Version	Types	Inst.	Concr.	Symb.	Fields	Rej./Unc.	NLOC	Files	Tests
Original code	7	7	21	36	11	10/3	458	17	not_run
After Change1	8	8	24	41	12	10/3	508	21	23/23
After Change2	9	9	28	48	14	11/3	586	25	30/30
After Change3	9	9	28	56	14	12/3	638	26	37/37

Symbols = unique class/interface/method symbols; Fields = injection-point fields mapped additionally; Rej./Unc. = rejected/uncertain candidates in that version; NLOC and files = production code (lizard 1.24.1; maximum cyclomatic number of any function is 5 in all versions, so complexity did not grow); Tests = passing tests of the sandbox harness (the baseline column was not run in its own commit; its 15 characterization tests were added in c9d7097 and pass on the unchanged production code).

3.2 CH01 - Account-type policies (overdraft for checking, exact-balance withdrawal)

Requirement. Business requirement: deposit bonus and withdrawal limits must depend on the account type, and a checking account may go into overdraft.
- Savings (CONTA_POUPANCA): deposits still receive the 0.5 bonus; withdrawal allowed only while amount <= balance (the original code rejected a withdrawal of exactly the balance - this boundary is changed on purpose).
- Checking (CONTA_CORRENTE): no bonus; withdrawal allowed while balance - amount >= -500.00 (overdraft limit).
- Amounts <= 0 are still rejected with HTTP 400; unknown accounts still 404.

AC	Acceptance criterion	Test
AC1	Checking with balance 100: withdraw 600 succeeds, balance becomes -500	Change1PolicyTest. ac1CheckingMayOverdrawUpToTheLimit
AC2	Checking with balance 100: withdraw 600.01 -> 400, balance unchanged	Change1PolicyTest. ac2CheckingCannotOverdrawBeyondTheLimit
AC3	Savings with balance 100: withdraw 100.01 -> 400, balance unchanged	Change1PolicyTest.ac3SavingsCannotOverdraw
AC4	Savings with balance 100: withdraw 100 succeeds, balance 0	Change1PolicyTest. ac4SavingsMayWithdrawExactlyTheBalance
AC5	Deposit of 5 gives +5.5 on savings and +5 on checking	Change1PolicyTest. ac5SavingsDepositBonusIsPreservedAndCheckingHasNone
AC6	Withdrawal of 0 or negative -> 400 for both types	Change1PolicyTest. ac6NonPositiveWithdrawalStillRejectedForBothTypes
AC7	Resolver returns the matching strategy; missing type -> 400; strategies interchangeable	Change1PolicyTest. resolverReturnsTheMatchingStrategyAndRejectsMissingType / policiesAreInterchangeableBehindTheInterface

Design smell / driver. Conditional on a type code with a magic number: AccountService.depositToAccount tested account.getType() == CONTA_POUPANCA and added the literal 0.5; the withdrawal rule balance > amount was one hard-coded rule for every type, so adding the overdraft would have added more type conditionals in two methods. **Technique.** Replace Conditional with Polymorphism (Replace Type Code with Strategy): extract interface AccountPolicy, move each branch into SavingsAccountPolicy / CheckingAccountPolicy, select through AccountPolicyResolver; Replace Magic Number with Constant (DEPOSIT_BONUS, OVERDRAFT_LIMIT). **Pattern effects.**

Pattern instance	Status	Note
Strategy: Account Type Policy	newly_introduced	Replaces the baseline conditional that was rejected as a Strategy candidate (AccountService savings-bonus if).
Transaction Script: Account Deposit and Withdrawal Script	modified	Still procedural, but its type-specific rules are delegated to the strategy.
Service Layer: Banking Use-Case Service Layer	modified	New collaborator policyResolver.
Dependency Injection: CDI Field Injection Wiring	extended	New bean AccountPolicyResolver and injection point.
DAO, DTO, Controller, Exception Translation	preserved	No participant changed.

Behaviour preservation. Tests: 14 of the 15 original characterization tests pass unchanged. The one that pinned the old boundary (withdrawOfExactlyTheBalanceIsRejectedInBaseline) was changed on purpose to withdrawOfExactlyTheBalanceIsAllowedAfterCh01 because AC4 changes that behaviour; savings deposit bonus, error codes and messages are covered by unchanged tests.

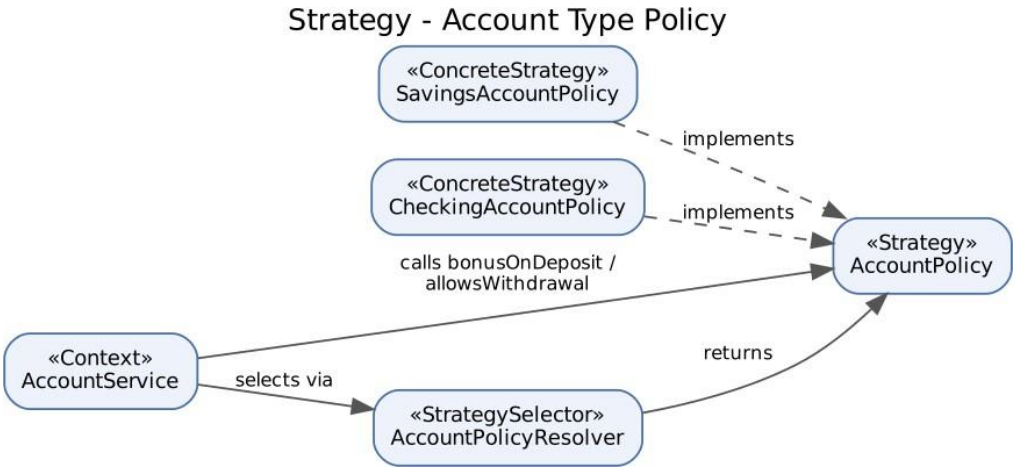


Figure 9: New pattern instance introduced by Change 1 (source: Part2/diagrams/after_change1/STRAT.dot).

3.3 CH02 - Transaction events: audit log and high-value alert observers

Requirement. Business requirement: every completed deposit or withdrawal must be recorded in an audit trail that can be read per account, and transactions of 10,000.00 or more must raise an alert, without the account service depending on the audit/alert code.
- AccountService publishes a TransactionEvent (account id, kind DEPOSIT/WITHDRAWAL, amount, balance after, timestamp) after the balance is saved.
- TransactionAuditLog stores events per account in order; GET /api/v1/accounts/*{id}*/audit returns them (404 for an unknown account).
- HighValueAlertObserver records and logs events with amount >= 10,000.00.
- Rejected operations (400/404) publish nothing.

AC	Acceptance criterion	Test
AC1	Deposit of 25 on balance 100 publishes one DEPOSIT event with balanceAfter 125	Change2ObserverTest. ac1DepositPublishesOneEventWithTheResultingBalance
AC2	Withdrawal publishes a WITHDRAWAL event	Change2ObserverTest. ac2WithdrawalPublishesAWithdrawalEvent
AC3	Failed deposit/withdraw/unknown account publish nothing	Change2ObserverTest. ac3FailedOperationsPublishNothing
AC4	Audit trail is per account and chronological	Change2ObserverTest. ac4AuditTrailIsPerAccountAndChronological
AC5	Audit endpoint returns 200 + trail; 404 for an unknown account	Change2ObserverTest. ac5AuditEndpointReturnsTheTrailAndNotFoundForUnknownAcco
AC6	Alert raised at exactly 10000.00, not at 9999.99	Change2ObserverTest. ac6HighValueTransactionsRaiseAnAlertAtTheThreshold
AC7	AccountService holds no reference to a concrete observer	Change2ObserverTest. ac7AccountServiceKnowsNoConcreteObserver

Design smell / driver. Preventive: audit and alerting would otherwise be written as direct calls inside depositToAccount/withdraw, giving the service a new dependency (and a new edit) for every reaction (Divergent Change / Shotgun Surgery risk). No existing smell was removed in this change. **Technique.** Replace direct calls with publish/subscribe (introduce Observer through CDI events); the event is an immutable record. **Pattern effects.**

Pattern instance	Status	Note
Observer: Transaction Audit and Alert Notification	newly_introduced	New event type, two concrete observers; AccountService is the subject (CDI Event).
Transaction Script	modified	deposit/withdraw also fire an event.
Service Layer	extended	New use case getAuditTrail.
Controller	extended	New system operation listAudit.

Pattern instance	Status	Note
Dependency Injection	extended	Event<TransactionEvent>, TransactionAuditLog injected; two observer beans.
Strategy, DAO, DTO, Exception Translation	preserved	No participant changed.

Behaviour preservation. Tests: all 23 tests of the previous version pass unchanged (30/30 in total); the only edits to existing test code are two additional beans registered in TestApp. Balances and HTTP statuses of deposit/withdraw are unchanged.



Figure 10: New pattern instance introduced by Change 2 (source: Part2/diagrams/after_change2/OBS.dot).

3.4 CH03 - Atomic funds transfer; business rules move into Account

Requirement. Business requirement: customers can transfer money between two accounts atomically.
- POST /api/v1/accounts/transfer with fromAccountId, toAccountId, amount.
- The source account applies its own policy (overdraft for checking, none for savings); the target is credited without the savings deposit bonus.
- Same account, missing ids and amount <= 0 -> 400; unknown account -> 404; insufficient funds -> 400. In every failure neither account changes.
- The operation is one transaction (service-level @Transactional) and publishes TRANSFER OUT and TRANSFER IN events.

Refactoring that enables it: the deposit/withdraw rules move from AccountService into the Account entity so that deposit, withdraw and transfer share one implementation.

AC	Acceptance criterion	Test
AC1	Transfer 30 from checking(100) to savings(50): 70 and 80 (no bonus)	Change3TransferTest. ac1TransferMovesTheAmountWithoutDepositBonus
AC2	Insufficient funds on savings source -> 400, both balances unchanged	Change3TransferTest. ac2InsufficientFundsChangesNeitherAccount
AC3	Checking source may use its overdraft (-500 / 650)	Change3TransferTest. ac3CheckingSourceMayUseItsOverdraft
AC4	Same account, amount <= 0, missing id -> 400, nothing changes	Change3TransferTest. ac4SameAccountAndNonPositiveAmountAndMissingIdsAreBadReq
AC5	Unknown source or target -> 404, nothing changes	Change3TransferTest. ac5UnknownAccountIsNotFoundAndNothingChanges
AC6	TRANSFER OUT and TRANSFER IN events with the new balances	Change3TransferTest. ac6TransferPublishesOutAndInEvents
AC7	Account enforces deposit/withdraw/credit rules by itself	Change3TransferTest. ac7AccountEnforcesItsOwnRulesWithoutTheService

Design smell / driver. Anemic Domain Model / Feature Envy: AccountService read and wrote Account.balance and applied all rules while Account only had accessors; the deposit and withdraw procedures also duplicated the load-validate-save-publish shape, and contained an unreachable else-branch (AccountDAO.findById never returns an empty Optional - it throws NotFoundException - so 'optionalAccount != null && isPresent()' was always true). **Technique.** Move Method (rules to Account), Extract Method (loadAccount, publish), Remove Dead Code / Replace Optional-null checks with orElseThrow. **Pattern effects.**

Pattern instance	Status	Note
Transaction Script: Account Deposit and Withdrawal Script	replaced	Rules moved into Account; remaining service methods are plain orchestration/CRUD.

Pattern instance	Status	Note
Domain Model: Account Balance Domain Model	newly_introduced	Replaces the script; Account.deposit/withdraw/credit hold the rules.
Strategy	modified	Context moves from AccountService to Account.
Observer	modified	Publishing centralised in AccountService.publish; new transfer event kinds.
Service Layer	modified	New transfer use case; deposit/withdraw/transfer are service-level transactions.
DTO	extended	TransferDTO.
Controller	extended	transfer operation.
Exception Translation	extended	Account and transfer raise BadRequestException; load-Account raises NotFoundException.
DAO, Dependency Injection	preserved	No participant changed.

Behaviour preservation. Tests: all 30 tests of the previous version pass unchanged (37/37 in total) - same balances, same statuses, same messages for deposit and withdraw. Not covered by any test: rollback of a failed transfer inside a real database transaction (requires Quarkus + a database; see README).

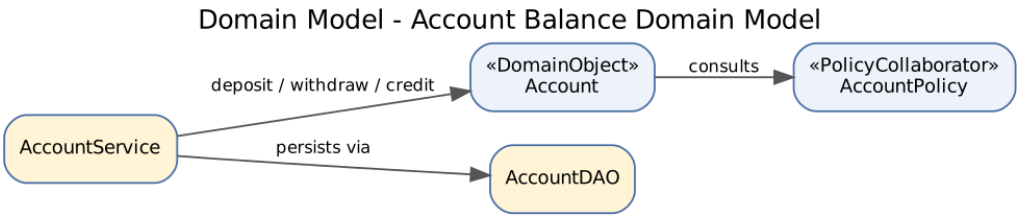


Figure 11: New pattern instance introduced by Change 3 (source: Part2/diagrams/after_change3/DM.dot).

3.5 Status of every instance across versions

Instance	Orig.	Ch.1	Ch.2	Ch.3
Account and User Persistence DAO	baseline	preserved	preserved	preserved
API Payload DTO Assembly	baseline	preserved	preserved	extended
Banking Use-Case Service Layer	baseline	modified	extended	modified
Account Deposit and Withdrawal Script	baseline	modified	modified	replaced
REST Use-Case Controller	baseline	preserved	extended	extended
CDI Field Injection Wiring	baseline	extended	extended	preserved
HTTP Error Translation	baseline	preserved	preserved	extended
Account Type Policy	–	newly_introduced	preserved	modified
Transaction Audit and Alert Notification	–	–	newly_introduced	modified
Account Balance Domain Model	–	–	–	newly_introduced

Interpretation. The layered patterns (DAO, DTO, Service Layer, Controller, Exception Translation) were preserved through all three changes: new behaviour was added around them rather than into them. Strategy removed a type-code conditional and made the next account type an additive change. The Observer decoupled reactions from the service and cost one new dependency (event injection) in it. Moving the rules into Account (Domain Model) retired the Transaction Script instance, so the number of types rose 7, 8, 9 and then stayed at 9 (Domain Model in, Transaction Script out). Limitation: the claims about transaction rollback of a failed transfer and about the HTTP/JPA wiring are not verified (no Quarkus run).

4 Part 3: Cross-language comparison

Ten pattern instances of the final application were re-implemented as small self-contained snippets in Java, Python, JavaScript (Node) and C++, 40 snippets in total, each with a runner that executes shared test cases (Part3/<problem>/test-cases/cases.txt). The snippets are *simplified, in-memory* versions: no frameworks, so CDI, JAX-RS and JPA behaviour is not exercised – the roles and collaborations are.

4.1 Selected instances and common specification

Pattern	Instance	Folder	Cases
Data Access Object	Account and User Persistence DAO	dao_account_persistence	7
Data Transfer Object	API Payload DTO Assembly	dto_user_assembly	5
Service Layer	Banking Use-Case Service Layer	service_layer_bank	6
Transaction Script	Account Deposit and Withdrawal Script	transaction_script_account	8
Controller	REST Use-Case Controller	controller_rest_use_case	5
Dependency Injection	CDI Field Injection Wiring	dependency_injection_wiring	7
Exception Translation	HTTP Error Translation	exception_translation_http	6
Strategy	Account Type Policy	strategy_account_policy	8
Observer	Transaction Audit and Alert Notification	observer_transaction_events	8
Domain Model	Account Balance Domain Model	domain_model_account	12

Fixture format: cmd1;cmd2 => expected output of the last command, numbers with two decimals; every runner prints RESULT passed=N total=M and the role check (Part3/evidence/role_check.txt) found every declared role in all 40 snippets (duck-typed languages have recorded exemptions for interfaces).

4.2 Results

Instance	Language	LOC	Max CC	Tests	Median ms
Account and User Persistence D	Java	42	2	7/7 passed	99.3
Account and User Persistence D	Python	33	2	7/7 passed	18.5
Account and User Persistence D	JavaScript	29	2	7/7 passed	49.9
Account and User Persistence D	C++	47	2	7/7 passed	4.2
API Payload DTO Assembly	Java	18	1	5/5 passed	105.4
API Payload DTO Assembly	Python	15	1	5/5 passed	16.1
API Payload DTO Assembly	JavaScript	13	1	5/5 passed	35.2
API Payload DTO Assembly	C++	15	1	5/5 passed	3.8
Banking Use-Case Service Layer	Java	30	3	6/6 passed	86.5
Banking Use-Case Service Layer	Python	32	2	6/6 passed	22.0
Banking Use-Case Service Layer	JavaScript	25	2	6/6 passed	34.2
Banking Use-Case Service Layer	C++	32	3	6/6 passed	3.2
Account Deposit and Withdrawal	Java	40	3	8/8 passed	87.3
Account Deposit and Withdrawal	Python	33	3	8/8 passed	17.7
Account Deposit and Withdrawal	JavaScript	26	3	8/8 passed	36.5
Account Deposit and Withdrawal	C++	38	3	8/8 passed	3.8
REST Use-Case Controller	Java	19	1	5/5 passed	124.3
REST Use-Case Controller	Python	17	1	5/5 passed	16.3
REST Use-Case Controller	JavaScript	8	1	5/5 passed	47.5
REST Use-Case Controller	C++	20	1	5/5 passed	3.2
CDI Field Injection Wiring	Java	32	4	7/7 passed	79.2
CDI Field Injection Wiring	Python	35	4	7/7 passed	18.4
CDI Field Injection Wiring	JavaScript	27	2	7/7 passed	36.0
CDI Field Injection Wiring	C++	36	4	7/7 passed	3.1
HTTP Error Translation	Java	29	3	6/6 passed	99.3
HTTP Error Translation	Python	27	3	6/6 passed	14.7

Instance	Language	LOC	Max CC	Tests	Median ms
HTTP Error Translation	JavaScript	24	3	6/6 passed	33.6
HTTP Error Translation	C++	29	3	6/6 passed	3.1
Account Type Policy	Java	34	3	8/8 passed	112.0
Account Type Policy	Python	35	3	8/8 passed	16.4
Account Type Policy	JavaScript	30	3	8/8 passed	48.6
Account Type Policy	C++	37	3	8/8 passed	3.4
Transaction Audit and Alert No	Java	46	3	8/8 passed	96.4
Transaction Audit and Alert No	Python	40	3	8/8 passed	17.4
Transaction Audit and Alert No	JavaScript	32	3	8/8 passed	34.0
Transaction Audit and Alert No	C++	54	5	8/8 passed	3.0
Account Balance Domain Model	Java	43	4	12/12 passed	96.4
Account Balance Domain Model	Python	48	4	12/12 passed	15.7
Account Balance Domain Model	JavaScript	36	4	12/12 passed	35.3
Account Balance Domain Model	C++	48	4	12/12 passed	3.9

Rules. LOC = non-blank non-comment lines of the snippet (runner excluded); Max CC = highest function cyclomatic number by lizard 1.24.1. Time = median of five timed runs after one warm-up run, compiled first, measured on the whole process (JVM/interpreter start-up plus reading the fixture file, compilation excluded; raw data in Part3/evidence/timing_raw.csv). Environment: shared 2-vCPU container (machine.txt), so the times are indicative only, not benchmarks.

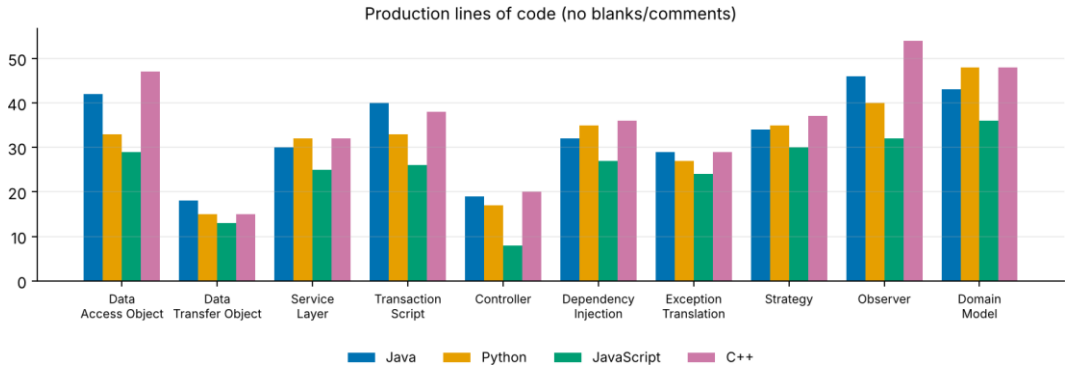


Figure 12: Source lines of code per problem and language.

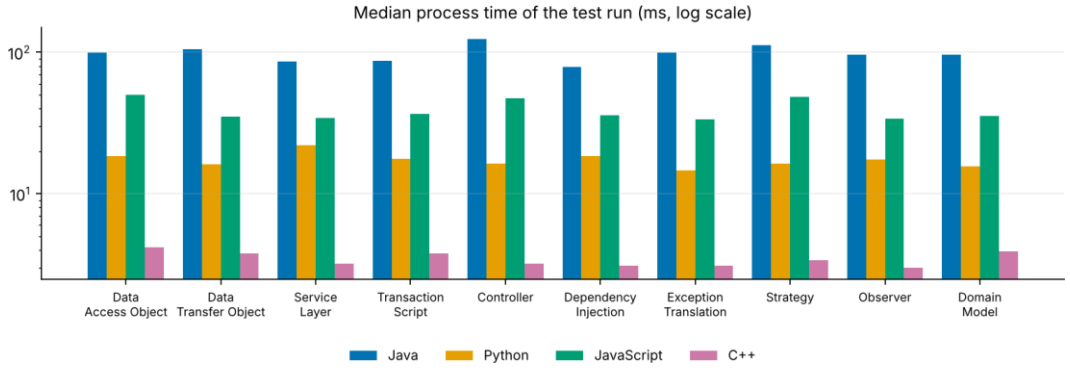


Figure 13: Median process time per problem and language (log scale).

Language	Median LOC	Median max CC	Median ms
Java	33.0	3.0	97.8
Python	33.0	3.0	16.9
JavaScript	26.5	2.5	35.6
C++	36.5	3.0	3.3

Comparison. Python and JavaScript snippets are the shortest because interfaces, getters and type declarations disappear (Strategy, Observer and DAO need no interface type; duck typing supplies the role). Java and C++ are longest: Java for explicit interfaces and exception classes, C++ for headers-in-file, constructors, ownership (smart pointers) and the standard-library plumbing. Cyclomatic complexity is nearly language independent, since it follows the pattern logic, not the syntax. Process time is dominated by start-up: native C++ is fastest, then Python, Node, and the JVM slowest – this says nothing about steady-state speed. Patterns whose purpose is compensating for static typing (Strategy as an interface, Dependency Injection through a container) shrink to a function or a constructor argument in the dynamic languages, while patterns tied to a framework (Exception Translation, Controller, DI container) lose the framework and keep only the structure. **Limitations.** The shared fixtures and the snippets come from the same LLM session (not independent); timings come from a shared container; snippets are simplified.

5 LLM usage records

The three prompts of each part belong to the three members. In this submission they were all run in three instances of Claude corresponding to their individual accounts: Member 1 (Hariprid V) = prompt M1, Member 2 (Pranav R Menon)
= prompt M2, Member 3 (Sreyas A) = prompt M3 (Part 1: presentation, business and persistence layer respectively; Parts 2 and 3: the corresponding prompt of each part). Files named output_ch0N.md are design notes written by the model; the generated code itself is the patch of each change. Every "evaluation" below was checked against the code or the tests, not taken from the model.

Part	Prompt	Output	Evaluation (verified)
Part 1	prompt_m1.txt	output_m1.md	Correct: 3 confirmed + 2 rejected candidates kept; all 18 file:line rows matched the source exactly and every behavioural claim passed the grep checks.
Part 1	prompt_m2.txt	output_m2.md	Partial: all 34 file:line rows matched; one claim was wrong - it said the only 'new' expressions create data objects, but exceptions are also created with new (claims_check C3). Corrected in the report; the DI conclusion is unaffected (no collaborator is created with new).
Part 1	prompt_m3.txt	output_m3.md	Correct: all 16 file:line rows matched; defects (Optional.of(null), dead catch, getAll null) confirmed by grep and by tests; the 2 uncertain candidates are kept uncertain because they cannot be proven in the sandbox.
Part 2	prompt_ch01.txt	output_ch01.md	Partial: production code compiled and all 23 tests passed on the first run, and the pattern analysis (Strategy introduced, Transaction Script modified) matches the code. One unannounced behaviour change was found in review: an account whose type is null could be credited before, but now gets HTTP 400 from AccountPolicyResolver.forType(null). One existing test had to be edited because the boundary change (AC4) was intended.
Part 2	prompt_ch02.txt	output_ch02.md	Partial: code compiled and 30/30 tests passed on the first run; claims about event content, per-account order, 400/404 publishing nothing and the 10000.00 threshold are confirmed by tests. Cannot confirm: that the audit trail stays correct if the surrounding database transaction rolls back after the event is fired (observers run synchronously inside the transaction; needs Quarkus + a database).

Part	Prompt	Output	Evaluation (verified)
Part 2	prompt_ch03.txt	output_ch03.md	Partial: code compiled and 37/37 tests passed on the first run; the Move Method refactoring preserved all earlier test results unchanged. The claim that the transfer is atomic is confirmed only for validation failures (nothing is persisted when a rule fails); rollback after a failure between the two DAO updates relies on @Transactional and was not executed. The unreachable-else-branch finding in the old deposit/withdraw code was confirmed by reading AccountDAO.findById.
Part 3	prompt_m1.txt	output_m1_generated_code.txt	Correct on execution: all 16 snippets (4 problems x 4 languages) compiled and passed 100% of their shared cases on the first run, and the role check found every declared role. Limitations: the fixtures were written by the same model from the Java behaviour, so they are not independent of the model; the snippets are simplified (in-memory, no framework), so framework behaviour (CDI, JAX-RS, JPA) is not exercised.
Part 3	prompt_m2.txt	output_m2_generated_code.txt	Correct on execution: all 12 snippets (3 problems x 4 languages) compiled and passed 100% of their shared cases on the first run, and the role check found every declared role. Limitations: the fixtures were written by the same model from the Java behaviour, so they are not independent of the model; the snippets are simplified (in-memory, no framework), so framework behaviour (CDI, JAX-RS, JPA) is not exercised.
Part 3	prompt_m3.txt	output_m3_generated_code.txt	Correct on execution: all 12 snippets (3 problems x 4 languages) compiled and passed 100% of their shared cases on the first run, and the role check found every declared role. Limitations: the fixtures were written by the same model from the Java behaviour, so they are not independent of the model; the snippets are simplified (in-memory, no framework), so framework behaviour (CDI, JAX-RS, JPA) is not exercised.

Human reviewer sign-off: Hariprid V, Pranav R Menon, Sreyas A.

6 Findings, contributions and references

6.1 Findings

Pattern mix: the baseline is a conventional layered design with 7 confirmed patterns; its weakest point is that the DAO abstraction is bypassed (services inject the concrete DAOs) and the business rules sit in the service (Transaction Script, anemic entity).

Defects found while verifying (not fixed, since they are outside the requested changes): UserDAO.findById can throw a NullPointerException and has a dead catch; getAll may return null; the Bean-Validation annotations are ineffective because the controllers lack @Valid; Account has no getId; the user password is stored and exposed as plain text; the template tests are stale.

Counting: with the rule that replaced instances are excluded, types went 7, 8, 9, 9 and instances 7, 8, 9, 9; concrete classes 21, 24, 28, 28; production NLOC 458, 508, 586, 638 with maximum CC constant at 5.

Behaviour changes made on purpose: CH01 allows withdrawing exactly the balance and returns 400 for a null account type.

Not verified: a real Quarkus build/test run, database transactions (rollback of failed transfers)

and HTTP wiring – run ./mvnw test in Part2/code before submission.

6.2 Contributions

Member	Contribution
Hariprid V	Prompt M1 in each part (Part 1: presentation/contract layer, Part 2: Change 1 policies, Part 3: first problem set); reviewed all patterns.
Pranav R Menon	Prompt M2 in each part (Part 1: business layer, Part 2: Change 2 observers, Part 3: second problem set); reviewed all patterns.
Sreyas A	Prompt M3 in each part (Part 1: persistence layer, Part 2: Change 3 transfer/domain model, Part 3: third problem set); reviewed all patterns.

6.3 References

1. M. Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002: Transaction Script, Service Layer, Data Transfer Object, Domain Model, https://martinfowler.com/eaaCatalog/.
2. Oracle, Core J2EE Patterns: Data Access Object, https://www.oracle.com/java/technologies/dataaccessobject.html.
3. E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns*, Addison-Wesley, 1994 (Strategy, Observer, Template Method).
4. C. Larman, *Applying UML and Patterns*, 3rd ed., 2004 (GRASP Controller).
5. Jakarta Contexts and Dependency Injection 4.0, https://jakarta.ee/specifications/cdi/4.0/jakarta-cdi-spec-4.0.html.
6. Jakarta RESTful Web Services 3.1, https://jakarta.ee/specifications/restful-ws/3.1/jakarta-restful-ws-spec-3.1.html.
7. Quarkus CDI reference, https://quarkus.io/guides/cdi-reference.
8. lizard code complexity analyser, https://github.com/terryyin/lizard.

7 Team folder and repository

Folder layout of the submission (generated listing, the large per-problem folders are summarised):

```
.
Part1
Part1/Part1_prompt.txt
Part1/Pattern_count.csv
Part1/evidence
Part1/llm
Part1/llm.csv
Part1/pattern_architecture
Part1/patterns.csv
Part2
Part2/Part2_prompt.txt
Part2/Pattern_count.csv
Part2/changes
Part2/code
Part2/diagrams
Part2/evidence
Part2/llm
Part2/llm.csv
Part2/pattern_evolution.csv
Part3
Part3/controller_rest_use_case
Part3/dao_account_persistence
Part3/dependency_injection_wiring
Part3/domain_model_account
```

Part3/dto_user_assembly
Part3/evidence
Part3/exception_translation_http
Part3/llm
Part3/observer_transaction_events
Part3/service_layer_bank
Part3/strategy_account_policy
Part3/transaction_script_account
README.md
data
data/code
data/test_result
data/test_result/README.md
data/test_result/baseline_tests
data/test_result/baseline_tests_run.log
data/test_result/harness
data/test_result/tools